Webpack Interview & Development Cheat Sheet

Part 3 — Advanced Configuration, Deployment, Debugging & Interview Revision

> **Primary reference:** Based on your uploaded Webpack course notes. Modern Webpack 5 additions are clearly identified.

Environment Variables

Why?

Avoid hardcoding values.

Instead of

```
const API_URL = "https://prod.api.com";
```

use

```
process.env.API_URL
```

DefinePlugin

```
plugins: [
  new webpack.DefinePlugin({
    "process.env.API_URL": JSON.stringify("https://api.example.com")
  })
]
```

Quick Reference

Concept	Notes
Purpose	Replace constants during compilation
Runtime?	❌ No
Compile time?	✅ Yes
Common use	API URL, feature flags, build version

Memory

> **DefinePlugin performs text replacement during compilation.**

---

dotenv

Typically

`.env`

`.env.development`

`.env.production`

Often combined with

`dotenv-webpack`

or custom configuration.

Multiple Configurations

Instead of one huge file

`webpack.config.js`

split into

`webpack.common.js`

`webpack.dev.js`

webpack.prod.js

Benefits

- Easier maintenance
- Less duplication
- Cleaner builds

---

webpack-merge

```
module.exports = merge(common, {
    mode: "production"
});
```

Think

Common

+

Production

=

Final Config

Interview

Why use webpack-merge?

Avoid duplicated configuration.

Externals

Suppose React comes from CDN.

Instead of bundling

```
React
```

Webpack assumes

```
window.React
```

Example

```
externals:{
   react:"React"
}
```

Benefits

- Smaller bundles
- Faster builds

Use carefully.

Module Federation ⭐ (Webpack 5)

Purpose

Share code between applications.

Example

```
Host App

↓

Remote App

↓

Shared Components
```

Use Cases

- Micro Frontends
- Enterprise portals
- Shared design systems

Interview

One of the most important Webpack 5 features.

Production Builds

Typical Pipeline

```
Source
↓
Loaders
↓
Plugins
↓
Optimization
↓
Chunks
↓
Assets
↓
dist/
```

Production Checklist

✅ Production mode

✅ Source Maps (hidden)

✅ Content Hash

✅ Tree Shaking

✅ SplitChunks

✅ Runtime Chunk

✅ Minification

✅ Compressed Assets

Deployment

Course covers several deployment strategies.

Platform	Use
GitHub Pages	Static sites
Surge	Quick demos
Amazon S3	Static hosting
Express	Dynamic applications
Heroku	Node deployment
Elastic Beanstalk	Production infrastructure

Static vs Dynamic Deployment

Static

`HTML`

`CSS`

`JS`

↓

Serve directly

Dynamic

```
Browser

↓

Node

↓

Database

↓

API
```

Webpack only builds assets.

Node serves application.

Express Integration

Development

```
Express

↓

webpack-dev-middleware

↓

Webpack
```

Production

```
Express

↓
```

dist/

↓

Browser

Memory

Development

→ Build every change

Production

→ Serve built files

---

webpack-dev-middleware

Purpose

Integrates Webpack into Express.

Benefits

- Memory builds
- Faster development
- Automatic recompilation

Debugging

Build Errors

Usually

- Missing loader
- Wrong regex
- Syntax errors

- Wrong import

Runtime Errors

Usually

- Wrong path
- Undefined exports
- Circular dependencies
- Missing assets

Useful CLI

```
npm run build
```

```
webpack --watch
```

```
webpack serve
```

```
webpack --mode production
```

Bundle Analysis

When bundle grows unexpectedly

↓

Run Bundle Analyzer

↓

Find

- duplicate packages
- huge libraries

- unnecessary imports

Security Considerations

Practice	Why
Never expose secrets	Bundles are public
Environment variables	Store server secrets outside bundle
Minify production	Harder to inspect
HTTPS assets	Prevent mixed content
Keep dependencies updated	Avoid vulnerabilities

Interview

Can secrets be hidden in Webpack?

No.

Anything bundled reaches the browser.

Common Errors

Error	Cause	Fix
Module not found	Wrong path	Check import
Unexpected token	Missing loader	Configure loader
CSS not loading	Missing css-loader/style-loader	Install both
Images missing	Wrong asset config	Asset Modules
Build slow	Huge dependencies	SplitChunks
Cache not updating	Missing content hash	Use `[contenthash]`
HMR not working	Wrong Dev Server config	Enable hot

Best Practices

Configuration

✅ Keep configs small.

✅ Split dev/prod configs.

✅ Use aliases.

Performance

✅ Lazy load routes.

✅ Split vendor code.

✅ Enable caching.

✅ Analyze bundles.

Assets

✅ Compress images.

✅ Prefer Asset Modules.

✅ Remove unused assets.

Development

✅ Enable HMR.

✅ Use Source Maps.

✅ Keep builds incremental.

Production

✅ Production mode.

✅ Minify.

✅ Tree shake.

✅ Content Hash.

Frequently Asked Interview Questions

Question	Short Answer
Why Webpack?	Module bundler and build tool.
Dependency Graph?	Graph of imported modules.
Entry?	Starting module.
Output?	Generated bundles.
Loader?	Transforms files.
Plugin?	Extends compilation.
Babel?	JavaScript transpiler.
css-loader?	Reads CSS.
style-loader?	Injects CSS.
Asset Modules?	Modern asset handling.
Dev Server?	Development web server.
HMR?	Replace changed modules only.
Code Splitting?	Multiple bundles.
Dynamic Import?	Async loading.
Lazy Loading?	Load when needed.
SplitChunks?	Shared dependency extraction.
Runtime Chunk?	Module loading runtime.
Tree Shaking?	Remove unused exports.
Side Effects?	Safe dead-code elimination.
Content Hash?	Cache busting.
DefinePlugin?	Compile-time constants.
HtmlWebpackPlugin?	Generates HTML.

Question	Short Answer
MiniCssExtractPlugin?	Extract CSS.
webpack-merge?	Merge configs.
Externals?	Exclude libraries.
Module Federation?	Share modules between apps.

One-Page Interview Revision

Build Pipeline

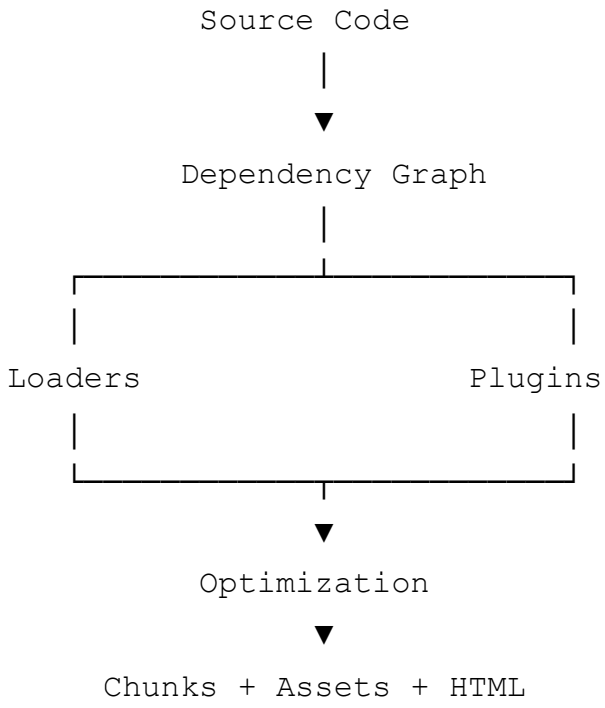
```
Source
    |
Entry
    |
Dependency Graph
    |
Loaders
    |
Plugins
    |
Optimization
    |
Chunks
    |
Assets
    |
Output
```

Remember These Pairs

Topic	Memory
Entry	Starting point
Output	Destination
Loader	Transform files

Topic	Memory
Plugin	Extend build
Babel	Transpile
Webpack	Bundle
css-loader	Read CSS
style-loader	Inject CSS
SplitChunks	Share dependencies
Dynamic Import	Async loading
Lazy Loading	Load later
Content Hash	Cache busting
Tree Shaking	Remove dead code
HMR	Replace changed modules
Source Maps	Debug bundles
DefinePlugin	Compile-time constants
Asset Modules	Modern asset handling

Webpack Mental Model (Remember Forever)



▼

dist/

> **One sentence to remember:**
>
> **Webpack starts at an entry file, builds a dependency graph, transforms files with loaders, extends the build with plugins, optimizes everything, and emits production-ready assets.**

Additional Modern Topics (Not Covered in the Course)

The course focuses on **Webpack 2** and introduces many concepts that remain relevant today. The following topics are important for **Webpack 5** but are not covered (or are only briefly mentioned) in the reference:

Topic	Why It Matters
Asset Modules	Replace `file-loader`, `url-loader`, and `raw-loader`.
Persistent Filesystem Cache	Dramatically faster rebuilds.
Module Federation	Enables Micro Frontends and shared modules across applications.
Deterministic Module IDs	Improves long-term browser caching.
`experiments`	Enables experimental features like top-level `await`.
CSS Modules	Component-scoped CSS for large applications.
PostCSS	Autoprefixing, modern CSS transformations, and optimization.
ESLint Integration	Catch code quality issues during builds.
Web Workers	Bundle worker scripts for background processing.
WebAssembly (WASM)	Import and bundle `.wasm` modules.

Congratulations!

Across these three parts, you've built a compact Webpack reference that complements your detailed course notes.

This cheat sheet is designed to help you:

- **Review concepts quickly before interviews**
- **Recall configuration patterns while building applications**
- **Jump into your primary notes only when you need deeper explanations**

With this reference, you should be able to refresh most Webpack concepts in just a few minutes before an interview or while working on a project.